Section 13

Section 12

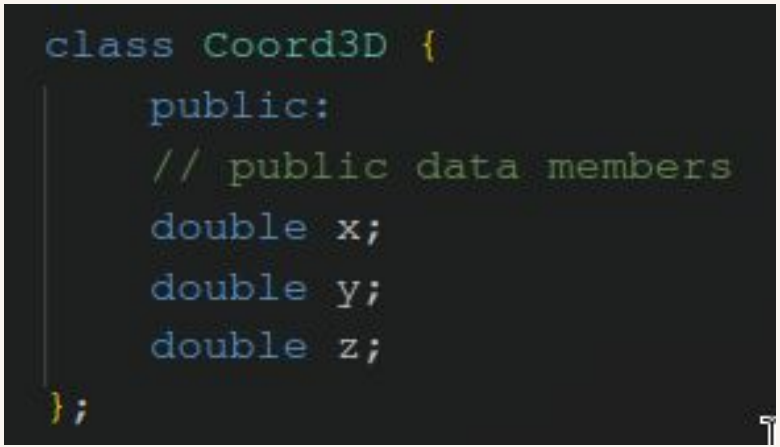
Pointers & Classes

Lab 9: https://maryash.github.io/135/labs/lab_09.html

Ryan Vaz

Introduction to basic Classes

Classes in C++ are user-defined compound data-types that can be used for grouping together several related variables. These “variables” inside a class are called fields (or data-members) of the classes.



Introduction to basic Classes

If you create a variable of a class, it can be referred to as an object.

```
int main {
    Coord3D pointP = {10, 20, 30};  // create a `Coord3D` object
    cout << pointP.x << endl;       // prints the value of data-member `x` which is 10
    pointP.y = 50;                  // set data-member `y` of `pointP` equal to 50
    cout << pointP.y << endl;       // will print 50
}
```

Class pointer

Whenever we need to tell someone the location of some place, we give them the address. We don't have to give them details about the whole location. The address can be written down in a small piece of paper. Similarly, instead of referring to an entire object or a large variable, using pointers can help us access much larger objects while passing it in a much smaller size.

```
int main {
    Coord3D pointP = {10, 20, 30};   // create a `Coord3D` object with x=10, y=20, z=30
    Coord3D* p = &pointP;            // pointer `p` points to `pointP` object
    Coord3D pointPCopy = *p;         // dereference p and make a copy of the object
    cout << pointPCopy.z << endl;    // prints 30
}
```

Introducing the -> operator

Dereferencing and accessing a classes data-members requires us to create a local copy of the object. What if we wanted to access the data-members directly from the pointer? In that case use -> operator:

```
int main {
    Coord3D pointP = {10, 20, 30};  // create a `Coord3D` object with x=10, y=20, z=30
    Coord3D* p = &pointP;           // pointer `p` points to `pointP` object
    cout << p->y << endl;           // prints 20
    p->y = p->z + 10;               // update the value of y data-member (30+10)
    cout << pointP.y << endl;       // prints 40
}
```

Automatic memory management

Normally, any variable “lives” only within the block where it is declared, and disappears once the program execution leaves this scope.

This type of memory management is called automatic. The program allocates memory for each variable when it enters the scope of the variable, and deletes that memory when leaving that scope.

This is how C++ handles local variables within functions and scope.

This can prove to be a problem…

Automatic memory management

The following function is responsible for creating a string variable with a poem and returning its memory address. This function will not work correctly:

```
string* createAPoem() {
    string poem = // making a string with a poem
    " Said Hamlet to Ophelia, \n"
    " I'll draw a sketch of thee, \n"
    " What kind of pencil shall I use? \n"
    " 2B or not 2B? \n";
    return &poem; // and returning its address
}
```

Since the variable poem exists locally inside the function, after exiting the function, the memory allocated for this string gets claimed and freed.

Even though we returned the address where the poem was, after the function exits, the address may be taken and used by some other part of your program, essentially overwriting it with some new value.

Dynamic memory allocation

We can fix this by allocating a chunk of memory for the poem so it would remain persistent in memory and would not be claimed by the program after the function exits. This is done using the new keyword.

```
string* createAPoemDynamically() {
    string* ppoem;          // declare a pointer to string to store the address
    ppoem = new string;     // <-- DYNAMICALLY ALLOCATE MEMORY
                            // for a poem string and store its address in the pointer
    *ppoem =                // put a poem there
    " If you know \n"
    " you know \n";
    return ppoem;           // return the address where the poem is
}
```

Dynamic memory allocation

The address of a dynamically allocated memory can be passed around, returned from a function, or stored in another variable, etc. The dynamically allocated memory will remain persistently in the computer memory throughout execution:

```
int main() {
    string * p;
    p = createAPoemDynamically();
    // The memory at the address p still stores the poem we put in it during the function call. Neat!
    // At any later moment, you can print it out:
    cout << *p;
    // You can also save the address into another pointer variable:
    string *p2 = p; // then both pointers, p and p2, will be pointing to the same poem
    cout << *p2;
}
```

Dynamic memory allocation

Dynamic memory allocation is powerful, almost too powerful. With great power, comes great responsibility. It stays in memory persistently and does not depend on scope for its lifetime, which means it can actually outlive the program. If the dynamically allocated variable is not manually released into the system, it will keep on taking up space in memory. With enough time, this will slow down your computer and cause it to crash!

Once a dynamically allocated variable is not needed anymore, we can use the keyword `delete` to get rid of that memory.

```
int *p = new int;          // allocate an integer dynamically
*p = 1234;                 // we are using it
cout << *p << endl;        // print out 1234
delete p;                  // once it's not needed, delete it:
```